

Regex Playbook

Regular Expressions in Python 3

A Complete Reference for Patterns, the re Module, Validation & Data Cleaning

Part of the Series:

End-to-End GenAI & Data Science: Production-Ready Engineering

- Introduction - What Regex Is & Why It Matters
 - Literal Characters, Wildcards & Anchors
 - Character Classes, Ranges & Negation
- Shorthand Classes - `\d` `\w` `\s` and Their Inverses
 - Groups, Alternation & Repetition Quantifiers
- The Python re Module - All 8 Functions Explained
 - Match Objects - `group()`, `start()`, `end()`, `span()`
 - Regex Flags - `re.I`, `re.M`, `re.S`
 - Lookahead & Lookbehind Assertions
- Real-World Applications - Validation, Cleaning & Redaction

SECTION 1 - THEORY

Chapter 1: Introduction to Regular Expressions

1.1 What Is a Regular Expression?

A Regular Expression (commonly abbreviated as regex or regexp) is a formal notation - a mini-language unto itself - for describing patterns in text. A regex pattern is a compact string of characters that encodes a search specification: when applied to a body of text, it identifies every substring that matches the described pattern. Regex is not unique to Python; it is a universal concept implemented across virtually every programming language, text editor, and command-line tool.

The term 'regular expression' has a precise mathematical origin: it refers to a pattern that can be recognised by a finite automaton (a type of abstract computational machine). In practice, modern regex engines extend the classical formal definition with features such as lookaheads, backreferences, and named groups, making them considerably more powerful than the strict theoretical definition.

"Regular expressions are the Swiss Army knife of text processing: compact, versatile, and requiring a manual on first use." - A practical Python engineer's observation

1.2 Why Learn Regex?

Regular expressions appear in virtually every domain of data science, software engineering, and GenAI engineering. The ability to construct, read, and debug regex patterns is a high-leverage skill that pays dividends across an enormous range of tasks:

Use Case	Example Application	Regex Role
Text Search	Find all log entries containing ERROR	Pattern: <code>r'ERROR\s+\w+'</code>
Input Validation	Ensure email addresses are well-formed	Pattern: email format regex
Information Extraction	Pull product codes from a document	Pattern: structured code format
Text Replacement	Censor credit card numbers in logs	Replace matched pattern with ****

Data Cleaning	Standardise phone number formats	Strip non-digits, reformat
Tokenisation	Split text on whitespace or punctuation	<code>re.split()</code> with regex delimiter
NLP Preprocessing	Remove punctuation, normalise whitespace	<code>re.sub()</code> with character class
GenAI Prompt Engineering	Extract structured fields from LLM output	Named group patterns

1.3 How Regex Works: The Engine

When Python's `re` module applies a regex pattern to a text string, it uses a backtracking finite automaton - an engine that reads the pattern and the input text character by character, attempting to find a configuration where the pattern aligns with a substring of the text. Understanding this process at a high level explains the behaviour of more advanced features like greedy vs lazy matching and lookaheads:

- * The engine starts at position 0 of the input string and attempts to match the pattern beginning at that position.
- * If the attempt succeeds, the match object is returned. If it fails, the engine advances one position in the input and tries again.
- * If the engine reaches the end of the input without a match, it returns `None`.
- * For complex patterns with multiple alternatives, the engine may explore many possible paths before settling on a match or concluding no match exists - a process called backtracking.

Python Import: All regex functionality in Python is provided by the `re` standard library module. It requires no installation: `import re` is sufficient. For large volumes of text, the third-party regex module (`pip install regex`) provides additional features and better Unicode support.

Chapter 2: Reading Regex - Fundamental Building Blocks

2.1 Literal Characters

The most basic regex element is a literal character - a character in the pattern that matches exactly that character (and only that character) in the input text. Regex is case-sensitive by default, so uppercase and lowercase letters are treated as distinct.

Example: Literal match

Pattern	Input Text	Result
sight	Pluralsight	Match - 'sight' found within 'Pluralsight'
Sight	Pluralsight	No match - 'S' does not equal 's' (case-sensitive)
Python	Python Regex Playbook	Match - 'Python' found at position 0

Literal characters form the backbone of most practical regex patterns: the complex patterns used in validation, extraction, and cleaning all consist of literal characters combined with special metacharacters.

2.2 The Wildcard (.) - Match Any Single Character

The dot `.` is the most fundamental metacharacter. It matches any single character except a newline character. It does not match zero characters, and it does not match two characters - exactly one character, any character except newline.

Pattern: `f.n`

Matches: <code>fun</code> - <code>fan</code> - <code>fin</code> - <code>fOn</code> - <code>f9n</code>
No Match: <code>fn</code> (no middle char) - <code>faan</code> (two middle chars)

The wildcard is useful when you know the structure of a pattern but not all the characters. For instance, matching any three-letter word starting with 'f' and ending with 'n'. In data cleaning, wildcards help construct flexible patterns for partially known text structures.

2.3 Anchors - Constraining Position

Anchors are zero-width assertions: they do not match any character, but instead assert something about the position in the string where the match can occur. The two fundamental anchors are the caret and the dollar sign:

Anchor	Name	Asserts
^	Start Anchor	The following pattern must appear at the very beginning of the string
\$	End Anchor	The preceding pattern must appear at the very end of the string
^...\$	Full-string match	Pattern must match the entire string from start to end

Start Anchor Example: `^Plural`

Matches: <code>Pluralsight</code> - <code>Plural</code>
--

No Match: Courseplural (Plural not at start) - not Plural

End Anchor Example: `sight$`

Matches: Pluralsight - oversight

No Match: sightseeing (sight not at end) - sight!

Data Science Application: Anchors are essential in validation regex. Without `^` and `$`, the pattern `r'd+'` would accept `'123abc'` (matching only the digits). Adding anchors - `r'^d+$'` - ensures the entire string consists only of digits. Always use anchors in validation patterns.

2.4 The Escape Character (\)

Many characters have special meaning in regex. To match one of these characters literally, prefix it with a backslash. In Python, always use raw strings (`r'...'`) for regex patterns to avoid Python's own backslash interpretation interfering with the regex engine's backslash interpretation.

```
# Without raw string - Python processes \. before regex sees it:
pattern = "\." # Python: \. is just . (escape sequence eaten by Python)

# With raw string - Python leaves \. intact for the regex engine:
pattern = r"\." # Regex sees: \. -> matches literal period character

# Practical example:
import re
text = 'I like regex.'
print(bool(re.search(r'regex\.', text))) # True - matches 'regex.'
print(bool(re.search(r'regex\.', 'I like regex!'))) # False - ! is not .
```

Golden Rule: Always write regex patterns as raw strings in Python: `r'pattern'`. This prevents the need to double-escape backslashes (writing `r'd+'` instead of `\\d+'`) and eliminates an entire class of subtle, hard-to-debug escaping bugs.

Chapter 3: Character Classes

3.1 What Is a Character Class?

A character class, written inside square brackets [...], specifies a set of characters, exactly one of which must appear at the corresponding position in the input. A character class is a more

precise alternative to the wildcard: instead of matching any character, it matches only the characters you specify.

Pattern	Matches	Explanation
<code>[ae]</code>	a or e	Exactly one character: either 'a' or 'e'
<code>[aeiou]</code>	Any vowel	One of the five English vowels
<code>[0123456789]</code>	Any digit	A digit - equivalent to <code>[0-9]</code>
<code>[a-z]</code>	Any lowercase letter	Range: 'a' through 'z'
<code>[A-Z]</code>	Any uppercase letter	Range: 'A' through 'Z'
<code>[a-zA-Z]</code>	Any letter	Both lower and uppercase
<code>[a-zA-Z0-9]</code>	Any letter or digit	Alphanumeric character
<code>[^abc]</code>	Any char EXCEPT a, b, c	Negated class: ^ inside [] negates

3.2 Character Ranges

A hyphen inside a character class denotes a range. The range is based on the ASCII/Unicode code point order. Common ranges and their meanings:

```
# Character range examples
import re

re.findall(r'[a-z]', 'Hello World 123') # ['e', 'l', 'l', 'o', 'o', 'r', 'l', 'd']
re.findall(r'[A-Z]', 'Hello World 123') # ['H', 'W']
re.findall(r'[0-9]', 'Hello World 123') # ['1', '2', '3']
re.findall(r'[a-zA-Z]', 'Hello World 123') # ['H', 'e', 'l', 'l', 'o', 'W', 'o', 'r', 'l', 'd']
re.findall(r'[a-zA-Z0-9]', 'Hi! 42.') # ['H', 'i', '4', '2']

# Negation: match everything EXCEPT letters and digits
re.findall(r'[^\a-zA-Z0-9]', 'Hi! 42.') # ['!', ' ', '.', '']
```

3.3 Negated Character Classes

A caret `^` as the very first character inside a class negates it: the class matches any single character that is NOT in the specified set. This is distinct from the caret as a start-anchor - position matters: `^` outside brackets is an anchor; `[^...]` is negation.

Negated character classes are particularly useful for extracting 'everything that is not a separator', removing punctuation, or validating that a string contains only permitted characters.

Chapter 4: Shorthand Character Classes

4.1 The Six Built-In Shorthand Classes

Python's regex engine provides six shorthand escape sequences that represent commonly needed character classes. These shorthands dramatically reduce pattern verbosity and are universally supported across regex flavours:

Shorthand	Equivalent Class	Matches	Common Use
<code>\d</code>	<code>[0-9]</code>	Any decimal digit (0-9)	Extract numbers from text
<code>\D</code>	<code>[^0-9]</code>	Any non-digit character	Remove non-numeric characters
<code>\s</code>	<code>[\t\n\r\f\v]</code>	Any whitespace character	Tokenise on whitespace
<code>\S</code>	<code>[^\t\n\r\f\v]</code>	Any non-whitespace character	Extract tokens
<code>\w</code>	<code>[a-zA-Z0-9_]</code>	Word character: letter, digit, underscore	Match identifiers
<code>\W</code>	<code>[^a-zA-Z0-9_]</code>	Non-word character	Split on non-word boundaries

4.2 Using Shorthands in Patterns

```
import re

text = 'Order #A4-B7: delivered on 2024-11-15 at 14:30'

# \d - find all digit sequences
re.findall(r'\d+', text)
# ['4', '7', '2024', '11', '15', '14', '30']

# \w+ - find all word tokens (letters, digits, underscores)
re.findall(r'\w+', text)
# ['Order', 'A4', 'B7', 'delivered', 'on', '2024', '11', '15', 'at', '14', '30']

# \s+ - split on one or more whitespace characters
re.split(r'\s+', 'Python  Regex  Playbook')
# ['Python', 'Regex', 'Playbook']

# Combining shorthands: match alphanumeric code followed by digit
re.findall(r'[aA]\d', 'A4 is valid, a7 too, but B3 is not')
# ['A4', 'a7']
```

Unicode Awareness: In Python 3, `\w`, `\d`, and `\s` are Unicode-aware by default. `\d` matches not only `[0-9]` but also Arabic-Indic digits, superscript digits, and other Unicode

numeric characters. Add `re.ASCII` (or `re.A`) flag to restrict matching to ASCII characters only - important for strict input validation.

Chapter 5: Groups, Alternation & Repetition

5.1 Capturing Groups - (...)

Parentheses serve two distinct purposes in regex: they group subpatterns into a single logical unit (enabling quantifiers to apply to the group as a whole), and they capture the matched text of the group for later retrieval via `match.group()` calls.

```
import re

# Groups enable quantifiers to apply to the whole group:
re.findall(r'(ha){2}', 'hahaha') # ['ha'] - group 'ha' repeated twice
re.findall(r'(ha){2,3}', 'hahaha') # ['ha'] - group repeated 2 to 3
times

# Groups capture matched text:
match = re.search(r'(\d{4})-(\d{2})-(\d{2})', '2024-11-15')
print(match.group(0)) # '2024-11-15' - entire match
print(match.group(1)) # '2024' - first group
print(match.group(2)) # '11' - second group
print(match.group(3)) # '15' - third group

# Named groups - (?P<name>pattern) - for readable extraction
match = re.search(r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})',
'2024-11-15')
print(match.group('year')) # '2024'
print(match.group('month')) # '11'
print(match.groupdict()) # {'year': '2024', 'month': '11', 'day':
'15'}
```

5.2 Alternation - (abc|def)

The pipe character inside a group implements alternation: it matches either the subpattern to its left or the subpattern to its right. The engine tries the left alternative first; if it fails, it backtracks and tries the right alternative.

```
import re

# Simple alternation
re.findall(r'cat|dog', 'I have a cat and a dog') # ['cat', 'dog']

# Alternation within a group - enables quantifiers
re.findall(r'(abc|def)+', 'abcdefabc') # ['abc'] - captures last group
match
```

```
# Alternative file extensions
re.findall(r'\w+\.(csv|json|parquet)', 'data.csv report.json model.pkl
notes.txt')
# ['csv', 'json'] - note: findall returns group content, not full match

# To get full match when group is present:
re.findall(r'\w+\.(?:csv|json|parquet)', 'data.csv report.json
model.pkl')
# ['data.csv', 'report.json'] - (?:...) is a non-capturing group
```

5.3 Quantifiers - Controlling Repetition

Quantifiers specify how many times the preceding element (character, class, or group) must appear for the pattern to match. They are among the most frequently used regex constructs:

Quantifier	Meaning	Example Pattern	Matches
+	One or more	<code>.\.txt</code>	data.txt, report_2024.txt - not .txt
*	Zero or more	<code>.*.txt</code>	.txt, data.txt, long_name.txt
?	Zero or one (optional)	<code>colou?r</code>	color, colour - makes 'u' optional
{n}	Exactly n times	<code>\d{4}</code>	2024, 1999 - exactly four digits
{n,}	n or more times	<code>\d{3,}</code>	100, 1234, 99999
{n,m}	Between n and m times	<code>\d{2,4}</code>	10, 100, 9999

5.4 Greedy vs. Lazy Quantifiers

By default, all quantifiers are greedy: they match as many characters as possible while still allowing the overall pattern to match. Adding a `?` after a quantifier makes it lazy (non-greedy): it matches as few characters as possible.

```
import re
html = '<b>bold text</b> and <i>italic</i>'

# Greedy: <.*> matches as much as possible - from first < to last >
re.findall(r'<.*>', html)
# ['<b>bold text</b> and <i>italic</i>'] - entire string between first
and last tags

# Lazy: <.*?> matches as little as possible - each individual tag
re.findall(r'<.*?>', html)
# ['<b>', '</b>', '<i>', '</i>'] - each tag separately

# Practical: extract content between XML/HTML tags
re.findall(r'<b>(.*?)</b>', html) # ['bold text']
```

GenAI Application: Greedy vs lazy matching is critical when parsing LLM-generated structured output. When extracting content between delimiters (JSON-like fields, XML tags, markdown code fences), always use lazy quantifiers: `r'```.*(?)```'` rather than `r'```.*```'` to capture individual blocks, not everything from the first to the last delimiter.

5.5 Real-World Pattern: Hex Colour Validation

CSS hex colours are an instructive case study that combines character classes, alternation, and exact-count quantifiers. A valid hex colour starts with # followed by exactly 3 or 6 hexadecimal digits (0-9 and a-f, case-insensitive):

```
import re

# Six-character hex: # followed by exactly 6 hex digits
pattern_6 = r'^#(\d|[A-Fa-f]){6}$'

re.fullmatch(pattern_6, '#FF1234') # Match
re.fullmatch(pattern_6, '#12ABEF') # Match
re.fullmatch(pattern_6, '#XY1234') # None - X and Y are not hex
re.fullmatch(pattern_6, '#FF12')   # None - only 4 hex digits

# Three or six character hex (shorthand or full):
pattern_3_or_6 = r'^#((\d|[A-Fa-f]){3}){1,2}$'

re.fullmatch(pattern_3_or_6, '#ABC')      # Match (shorthand)
re.fullmatch(pattern_3_or_6, '#AABBCC')   # Match (full)
re.fullmatch(pattern_3_or_6, '#AABB')     # None - 4 hex digits, invalid

# Cleaner alternative using [0-9a-fA-F]:
pattern_clean = r'^#[0-9a-fA-F]{3}(?:[0-9a-fA-F]{3})?$'
# Explanation: # + 3 hex chars + optionally 3 more hex chars
```

Chapter 6: The Python re Module

6.1 Module Overview

Python's re module is the standard library implementation of a regular expression engine. It provides eight primary functions and a compile() mechanism for creating reusable compiled pattern objects. All functions share the same first two arguments: the pattern string and the input string. Optional flags modify matching behaviour.

Function	Returns	Primary Use
re.search(pattern, string)	Match object or None	Find first match anywhere in string
re.match(pattern, string)	Match object or None	Match only at beginning of string

<code>re.fullmatch(pattern, string)</code>	Match object or None	Entire string must match pattern
<code>re.findall(pattern, string)</code>	List of strings	Find all non-overlapping matches
<code>re.finditer(pattern, string)</code>	Iterator of match objects	Find all matches (memory-efficient)
<code>re.sub(pattern, repl, string)</code>	New string	Replace matches with repl
<code>re.subn(pattern, repl, string)</code>	(new string, count) tuple	Replace and return count
<code>re.split(pattern, string)</code>	List of strings	Split string on matches
<code>re.escape(string)</code>	Escaped string	Escape all special regex characters
<code>re.compile(pattern)</code>	Pattern object	Compile pattern for repeated use

6.2 `re.search()` - Find First Match Anywhere

`re.search()` scans through the string from left to right and returns a Match object for the first location where the pattern matches any substring. If no match is found anywhere, it returns `None`. Unlike `re.match()`, it does not require the match to start at the beginning of the string.

```
import re

text = 'Python Regex Playbook'

# Found:
result = re.search('Regex', text)
print(result)          # <re.Match object; span=(7, 12), match='Regex'>
print(result.group())  # 'Regex'
print(result.span())   # (7, 12)

# Not found:
result = re.search('Java', text)
print(result)          # None

# Safe usage pattern:
if match := re.search(r'\d+', 'Product ID: 4721'):
    print(f'Found number: {match.group()}') # Found number: 4721
else:
    print('No number found')
```

6.3 `re.match()` - Match at Beginning Only

`re.match()` attempts to apply the pattern only at position 0 (the very start) of the string. If the pattern matches the beginning, a Match object is returned. If the string begins with something other than the pattern, `None` is returned - even if the pattern occurs later in the string.

```
import re

re.match('Python', 'Python Regex') # Match - 'Python' is at position 0
```

```
re.match('Regex', 'Python Regex') # None - 'Regex' is not at position
0

# IMPORTANT: re.match does NOT match at the end implicitly
# This matches 'Py' at the start - it does not require full match:
re.match(r'Py', 'Python is great') # Match object for 'Py'

# For full string match, use re.fullmatch:
re.fullmatch(r'Python', 'Python') # Match
re.fullmatch(r'Python', 'Python Regex') # None - string is longer than
pattern
```

Function	Matches	Returns Match When
re.match()	Start of string only	Pattern aligns with start of string
re.search()	Anywhere in string	Pattern found anywhere in string
re.fullmatch()	Entire string	Pattern covers entire string exactly

6.4 re.findall() - Find All Matches

re.findall() returns a list of all non-overlapping matches of the pattern in the string, from left to right. When the pattern contains capturing groups, it returns a list of the groups' contents rather than the full matches:

```
import re

# No groups: returns list of full match strings
re.findall(r'\d', 'A1B2C3') # ['1', '2', '3']
re.findall(r'\d+', 'Age: 25, Score: 98') # ['25', '98']

# Multiple groups: returns list of tuples
re.findall(r'(\w+)= (\d+)', 'a=1 b=2 c=3')
# [('a', '1'), ('b', '2'), ('c', '3')]

# Practical: extract all email addresses from a document
text = 'Contact alice@example.com or bob@test.org for more info.'
emails = re.findall(r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}',
text)
print(emails) # ['alice@example.com', 'bob@test.org']
```

6.5 re.sub() and re.subn() - Replace Text

re.sub() replaces every non-overlapping match of the pattern with the replacement string and returns the modified string. re.subn() does the same but returns a tuple of (new_string, number_of_substitutions_made):

```
import re

# Basic replacement
re.sub(r'\d', '*', 'A1B2C3')      # 'A*B*C*'
re.subn(r'\d', '*', 'A1B2C3')    # ('A*B*C*', 3)

# Replace with a function (dynamic replacement)
def double_digit(match):
    return str(int(match.group()) * 2)

re.sub(r'\d', double_digit, 'A1B2C3')  # 'A2B4C6'

# Backreferences in replacement: \1 refers to first group
re.sub(r'(\d{4})-(\d{2})-(\d{2})', r'\3/\2/\1', '2024-11-15')
# '15/11/2024' - re-orders date components

# Limit replacements with count parameter
re.sub(r'\d', '*', 'A1B2C3', count=2)  # 'A*B*C3' - only first 2 replaced
```

6.6 re.split() - Split on a Pattern

`re.split()` splits the string at every position where the pattern matches, returning a list of the pieces between matches. It is a more powerful version of `str.split()` because the delimiter can be any regex pattern:

```
import re

# Split on one or more whitespace characters
re.split(r'\s+', 'Python  Regex  Playbook')
# ['Python', 'Regex', 'Playbook']

# Split on commas, semicolons, or colons (any of these delimiters)
re.split(r'[,:;]', 'a,b;c:d')  # ['a', 'b', 'c', 'd']

# With capturing group: keep the delimiter in the output
re.split(r'(\s+)', 'Python Regex Playbook')
# ['Python', ' ', 'Regex', ' ', 'Playbook'] - includes spaces as
elements

# Practical: parse a CSV-like line handling multiple delimiters
line = 'Alice  , 30 ; engineer | Python'
fields = re.split(r'\s*[,:;|]\s*', line)  # ['Alice', '30', 'engineer',
'Python']
```

6.7 re.escape() - Escaping User Input

`re.escape()` takes a string and returns a version in which all characters that would be interpreted as regex metacharacters are prefixed with a backslash. This is essential when incorporating user-supplied text into a regex pattern.

```
import re
```

```
user_input = 'file.txt'
# BAD: re.search('file.txt', text) - '.' is a wildcard, matches
# 'fileXtxt' too

# GOOD: escape user input before embedding in pattern
safe_pattern = re.escape(user_input)
print(safe_pattern)      # 'file\\.txt'
re.search(safe_pattern, 'file.txt')    # Match - searches for literal
# 'file.txt'
re.search(safe_pattern, 'fileXtxt')    # None - dot is not a wildcard
```

6.8 re.compile() - Reuse Patterns Efficiently

When the same pattern is applied many times (iterating over a large dataset, processing millions of log lines, or applying validation repeatedly), compiling the pattern once and reusing the compiled object is significantly faster than recompiling on every call:

```
import re

# With compile: compiled once, used many times
date_pattern = re.compile(r'\d{4}-\d{2}-\d{2}')

for record in records:
    if date_pattern.search(record):    # no recompilation
        ...

# Compiled pattern objects expose the same methods as the re module:
pattern = re.compile(r'\d+')
pattern.search('Age 25')              # equivalent to re.search(r'\d+', 'Age
25')
pattern.findall('A1B2C3')             # equivalent to re.findall(r'\d+',
'A1B2C3')
pattern.sub('*', 'A1B2C3')            # equivalent to re.sub(r'\d+', '*',
'A1B2C3')

# Store pattern and flags together:
ci_pattern = re.compile(r'python', re.IGNORECASE)
ci_pattern.findall('Python is PYTHON') # ['Python', 'PYTHON']
```

Chapter 7: Match Objects

7.1 What Is a Match Object?

The functions `re.search()`, `re.match()`, and `re.fullmatch()` return a `re.Match` object when they find a match, and `None` when they do not. The Match object encapsulates all information about the match: the matched text, its position in the input string, and the contents of any capturing groups.

Method / Attribute	Returns	Description
--------------------	---------	-------------

<code>match.group()</code>	str	The entire matched substring (equivalent to <code>group(0)</code>)
<code>match.group(n)</code>	str	Content of the nth capturing group
<code>match.group('name')</code>	str	Content of a named capturing group
<code>match.groups()</code>	tuple of str	All capturing group contents as a tuple
<code>match.groupdict()</code>	dict	Named groups as {name: content} dictionary
<code>match.start()</code>	int	Start position of the match in the input string
<code>match.end()</code>	int	End position (exclusive) of the match
<code>match.span()</code>	tuple (int, int)	(start, end) as a tuple
<code>match.string</code>	str	The original input string passed to the function
<code>match.re</code>	Pattern object	The compiled pattern that produced the match

7.2 Match Object in Practice

```
import re

# Basic match object properties
match = re.search(r'\d+', 'Age 25')
print(match.group())    # '25'
print(match.start())    # 4
print(match.end())      # 6
print(match.span())     # (4, 6)

# Named groups
match = re.search(
    r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2}) ',
    'Date: 2024-11-15'
)
print(match.group('year'))    # '2024'
print(match.groupdict())     # {'year': '2024', 'month': '11', 'day': '15'}

# Using span() for text manipulation
text = 'Product code: AB12-CD34'
match = re.search(r'[A-Z]{2}\d{2}-[A-Z]{2}\d{2}', text)
start, end = match.span()
redacted = text[:start] + '[REDACTED]' + text[end:]
print(redacted)             # 'Product code: [REDACTED]'
```

Chapter 8: Regex Flags

8.1 What Are Flags?

Regex flags modify the behaviour of the matching engine globally for a given function call. They can be passed as the third argument to most re functions, or set inline using the (?flag) syntax within the pattern itself. Python provides both verbose long-form names and short symbolic constants for each flag:

Flag (short)	Flag (long)	Effect
re.I	re.IGNORECASE	Make the pattern case-insensitive
re.M	re.MULTILINE	^ and \$ match start/end of each line (not just string)
re.S	re.DOTALL	Make . match newlines as well as other characters
re.X	re.VERBOSE	Allow whitespace and # comments in patterns
re.A	re.ASCII	Make \w, \d, \s match ASCII-only (not Unicode)
re.L	re.LOCALE	Make \w, \d, \s locale-dependent (deprecated, avoid)

8.2 re.IGNORECASE - Case-Insensitive Matching

```
import re

# Without flag: case-sensitive
re.search('python', 'Python is great')    # None - 'p' != 'P'

# With re.I flag: case-insensitive
re.search('python', 'Python is great', re.I)    # Match
re.findall('python', 'Python PYTHON python', re.IGNORECASE)
# ['Python', 'PYTHON', 'python']

# Inline flag syntax (embeds flag in pattern itself):
re.search('(?i)python', 'Python is great')    # Match - (?i) = re.I
```

8.3 re.MULTILINE - Line-by-Line Anchoring

```
import re

log_text = 'ERROR: connection failed\nINFO: retrying...\nERROR: timeout occurred'

# Without re.M: ^ matches only start of entire string
re.findall(r'^ERROR.*', log_text)
# ['ERROR: connection failed'] - only the very first line

# With re.M: ^ matches start of EVERY line
re.findall(r'^ERROR.*', log_text, re.MULTILINE)
# ['ERROR: connection failed', 'ERROR: timeout occurred']

# Combining flags with bitwise OR:
```

```
re.findall(r'^error.*', log_text, re.MULTILINE | re.IGNORECASE)
# ['ERROR: connection failed', 'ERROR: timeout occurred']
```

8.4 re.DOTALL - Match Newlines with .

```
import re

html = '<div>\n  <p>Some text</p>\n</div>'

# Without re.S: . does not match newline
re.search(r'<div>(.*?)</div>', html)  # None - content spans multiple
lines

# With re.S: . matches everything including newlines
match = re.search(r'<div>(.*?)</div>', html, re.DOTALL)
print(match.group(1))  # '\n  <p>Some text</p>\n'
```

8.5 re.VERBOSE - Readable Complex Patterns

Complex regex patterns quickly become unreadable walls of symbols. The `re.VERBOSE` flag allows whitespace and Python-style comments to be added to a pattern, dramatically improving readability for long patterns:

```
import re

# Same pattern with re.VERBOSE - commented and formatted:
email_verbose = re.compile(r'''
    ^                # Start of string
    [a-zA-Z0-9._%+-]+ # Local part: letters, digits, and allowed
symbols             # symbols
    @                # The @ separator
    [a-zA-Z0-9.-]+   # Domain name
    \.               # Literal dot
    [a-zA-Z]{2,}     # Top-level domain (2 or more letters)
    $                # End of string
''', re.VERBOSE)

email_verbose.fullmatch('user@example.com')  # Match
email_verbose.fullmatch('invalid-email')    # None
```

Chapter 9: Lookahead and Lookbehind Assertions

9.1 What Are Lookaround Assertions?

Lookahead and lookbehind assertions (collectively called lookarounds) are zero-width assertions: like anchors, they do not consume characters from the input string. Instead, they assert something about the characters immediately before or after the current position.

Syntax	Name	Matches current position when...
--------	------	----------------------------------

(?=...)	Positive Lookahead	The following characters match the pattern
(?!...)	Negative Lookahead	The following characters do NOT match the pattern
(?<=...)	Positive Lookbehind	The preceding characters match the pattern
(?<!=...)	Negative Lookbehind	The preceding characters do NOT match the pattern

9.2 Lookbehind - What Came Before

Positive lookbehind (**?<=...**) asserts that the pattern inside the lookbehind appears immediately before the current position. The lookbehind content is not included in the match:

```
import re

text = 'aB3cD4eF5gH6'

# Find uppercase letters preceded by a lowercase letter:
pattern = r'(?<=[a-z])[A-Z]'
matches = re.findall(pattern, text)
print(matches)    # ['B', 'D', 'F', 'H']
# 'B' matched because 'a' (lowercase) precedes it
# The 'a' is NOT included in the match - only 'B' is returned

# Practical: extract values that follow a known label
# 'Price: $99.99' - extract the number after '$'
re.findall(r'(?<=\\$)[\\d.]+', 'Price: $99.99, Sale: $49.50')
# ['99.99', '49.50'] - $ not included in match
```

9.3 Lookahead - What Comes Next

Positive lookahead (**?=...**) asserts that the pattern inside appears immediately after the current position. Combined with a lookbehind, it enables selecting characters based on both their left and right context:

```
import re

text = 'aB3cD4eF5gH6'

# Find uppercase preceded by lowercase AND followed by a digit:
pattern = r'(?<=[a-z])[A-Z](?=\\d)'
matches = re.findall(pattern, text)
print(matches)    # ['B', 'D', 'F', 'H']

# Extract words followed by a colon (dictionary keys in text)
text2 = 'Name: Alice, Age: 30, Score: 95'
keys = re.findall(r'\\w+(?=:)', text2)
print(keys)    # ['Name', 'Age', 'Score']

# Extract passwords that MUST contain a digit (positive lookahead):
has_digit = re.compile(r'^(?=.*\\d).+$')
```

```
bool(has_digit.match('password123')) # True - has digit
bool(has_digit.match('password'))    # False - no digit
```

Data Science Application: Lookarounds are invaluable for structured text extraction. When parsing LLM output, log files, or scraped web content, you often want to extract a value that follows a known label ('Score: 95') or precedes a known suffix ('95%'). Lookarounds let you match exactly the value you want, without the label or suffix polluting the match.

Chapter 10: Real-World Applications

10.1 Searching Patterns in Files

A common production task is extracting structured codes (product IDs, order numbers, error codes, tracking numbers) from large text files such as logs, reports, or exported data. The `re.findall()` function with file I/O makes this a compact, efficient operation:

```
import re

with open('sample.txt', 'r', encoding='utf-8') as f:
    text = f.read()

pattern = r'\b[A-Z]{2}\d{2}-[A-Z]{2}\d{2}\b|\b\d{3}-\d{3}\b'
matches = re.findall(pattern, text)
print('All product codes:', matches)

# Pattern breakdown:
# \b          - word boundary (ensures we don't match inside longer
strings)
# [A-Z]{2}    - exactly two uppercase letters
# \d{2}       - exactly two digits
# -          - literal hyphen
# |          - OR
# \b\d{3}-\d{3}\b - three digits, hyphen, three digits
```

10.2 User Input Validation

Input validation is one of the most critical applications of regex in production systems. Validating before processing prevents downstream failures, security vulnerabilities, and corrupted data.

```
import re

# Username validation: 4-15 characters, only letters, digits, underscores
USERNAME_PATTERN = re.compile(r'^[A-Za-z0-9_]{4,15}$')

def validate_username(username: str) -> bool:
    return bool(USERNAME_PATTERN.fullmatch(username))
```

```
print(validate_username('alice_99'))    # True
print(validate_username('ab'))          # False - too short
print(validate_username('alice smith')) # False - space not allowed

# Password strength validation
def is_strong_password(password: str) -> bool:
    checks = [
        re.search(r'.{8,}', password),          # minimum length
        re.search(r'[A-Z]', password),          # uppercase required
        re.search(r'\d', password),             # digit required
        re.search(r'[@#$%^&*()_+\-=]', password), # special char
    ]
    return all(checks)

print(is_strong_password('P@ssw0rd!')) # True
print(is_strong_password('password'))  # False - no uppercase or digit
```

10.3 Email Validation

Email validation is perhaps the most widely implemented regex pattern. A production-grade email regex must balance thoroughness with practicality:

```
import re

EMAIL_PATTERN = re.compile(r'''
    ^                # Start of string
    [a-zA-Z0-9._%+\-]+ # Local part: letters, digits, . _ % + -
    @                # @ separator (exactly one)
    [a-zA-Z0-9.\-]+   # Domain name: letters, digits, . -
    \.                # Literal dot before TLD
    [a-zA-Z]{2,}      # Top-level domain (2+ letters: com, org,
io)
    $                # End of string
''', re.VERBOSE)

def is_valid_email(email: str) -> bool:
    return bool(EMAIL_PATTERN.fullmatch(email))

valid_emails = ['test@gmail.com', 'user.name+tag@example.co.uk',
'a@b.io']
invalid_emails = ['missing-at.com', '@nodomain.com', 'double@@test.com']
```

10.4 Data Cleaning with Regex

Data cleaning is the most quantitatively impactful application of regex in data science. Raw datasets frequently contain inconsistent formatting that must be normalised before analysis or model training:

```
import re

# Standardise phone numbers
```

```
def normalise_phone(phone: str) -> str:
    digits_only = re.sub(r'\D', '', phone)      # remove everything
except digits
    if len(digits_only) == 11 and digits_only.startswith('1'):
        digits_only = digits_only[1:]
    return digits_only if len(digits_only) == 10 else None

print(normalise_phone('(123) 456-7890')) # '1234567890'
print(normalise_phone('+1 123 456 7890')) # '1234567890'
print(normalise_phone('123.456.7890'))    # '1234567890'

# Normalise whitespace
def normalise_whitespace(text: str) -> str:
    return re.sub(r'\s+', ' ', text).strip()

print(normalise_whitespace('Python \t Regex\n Playbook '))
# 'Python Regex Playbook'

# Extract and reformat dates
def us_to_iso_date(text: str) -> str:
    return re.sub(r'(\d{2})/(\d{2})/(\d{4})', r'\3-\1-\2', text)

print(us_to_iso_date('Event on 11/15/2024'))
# 'Event on 2024-11-15'
```

10.5 Redacting Sensitive Information

In data engineering, ML pipelines frequently process data that may contain sensitive information that must be redacted before the data is stored, logged, or used in training:

```
import re

# Redact credit card numbers - Format: NNNN-NNNN-NNNN-NNNN
def redact_credit_cards(text: str) -> str:
    return re.sub(r'\d{4}-\d{4}-\d{4}-\d{4}', '****-****-****-****',
text)

text = 'My credit card is 1234-5678-9012-3456. Expiry: 11/26.'
print(redact_credit_cards(text))
# 'My credit card is ****-****-****-****. Expiry: 11/26.'

# Redact email addresses
def redact_emails(text: str) -> str:
    return re.sub(
        r'[a-zA-Z0-9._%+\\-]+@[a-zA-Z0-9.\\-]+\\. [a-zA-Z]{2,}',
        '[EMAIL REDACTED]',
        text
    )

print(redact_emails('Contact alice@example.com for details. '))
# 'Contact [EMAIL REDACTED] for details.'

# Redact API keys (40-char hex strings)
def redact_api_keys(text: str) -> str:
```

```
return re.sub(r'[0-9a-fA-F]{40}', '[API_KEY]', text)
```

SECTION 2 - PRACTICAL & CODING

Chapter 11: Hands-On Notebook - Regex in Practice

The following cells form a complete, executable Jupyter notebook. Each cell builds on the previous, progressing from basic pattern construction through real-world data processing tasks. Execute cells sequentially for best results.

Cell 1 - Environment Setup & Regex Basics

```
# Cell 1: Setup and first regex experiment
import re

pattern = 'e'
text    = 'Python Regex Playbook'

match = re.search(pattern, text)
print(f'Pattern   : {pattern!r}')
print(f'Text      : {text!r}')
print(f'Match      : {match.group()!r}')
print(f'Position   : {match.span()!r}')

# findall: all occurrences
all_e = re.findall('e', text)
print(f"All 'e' occurrences : {all_e}")

# Case sensitivity demonstration
print('\n--- Case Sensitivity ---')
print(f'Search e  : {re.findall("e", text)!r}')
print(f'Search E  : {re.findall("E", text)!r}')
print(f'Ignore case : {re.findall("e", text, re.I)!r}')

# Wildcard demonstration
sentences = ["I'm a fan.", "I'm a fin.", "I'm a fun.", "I'm a fn."]
pattern_dot = r'f.n'
for sentence in sentences:
    result = re.search(pattern_dot, sentence)
    status = f'Match: {result.group()!r}' if result else 'No match'
    print(f' {sentence!r:25s} -> {status!r}')
```

Explanation

This opening cell establishes the fundamental `re.search()` and `re.findall()` workflow. Several observations are critical for developing correct regex intuition:

- * **re.search() vs re.findall():** search() returns the first match as a rich Match object (with span, group, position). findall() returns a plain list of all matching strings - useful for census-style extraction, but losing position information. Choose based on whether you need position data.
- * **Case sensitivity is the default:** 'e' and 'E' are treated as completely different characters. The re.I flag makes the matching engine case-insensitive for the entire call. Inline (?i) achieves the same effect within the pattern string.
- * **The wildcard (.) and the 'fn' no-match:** 'f.n' requires exactly one character between 'f' and 'n'. 'fn' has zero characters between them - the dot has no character to match, so no match occurs. This demonstrates that '.' means 'exactly one of any character', not 'zero or one of any character'.

Cell 2 - Character Classes, Anchors & Escaping

```
# Cell 2: Character classes, anchors, and proper escaping
import re

# Character classes
test_strings = ['a4', 'A4', 'b7', 'A7', 'a9', 'B3', 'c0']
pattern = r'[aA]\d'

for s in test_strings:
    result = re.search(pattern, s)
    print(f' {s!r:6s} -> {"Match" if result else "No match"}')

# Anchor demonstration
words = ['Pluralsight', 'Plural', 'notPlural', 'InsightPlural']
for word in words:
    start = re.search(r'^Plural', word)
    end = re.search(r'sight$', word)
    print(f' {word!r:20s} ^Plural: {"Y" if start else "N"} sight$: {"Y" if end else "N"}')

# re.escape for user-supplied input
user_term = 'file.txt'
safe = re.escape(user_term)
print(f'User input   : {user_term!r}')
print(f'Escaped      : {safe!r}')
print(f'Searches for literal dot: {bool(re.search(safe, "file.txt"))}')
print(f'Rejects fileXtxt      : {not bool(re.search(safe, "fileXtxt"))}')
```

Explanation

This cell provides hands-on demonstrations of the three constructs that form the backbone of most practical regex patterns:

- * **Character class [aA]\d:** Matches exactly one character from the set {a, A}, followed by exactly one digit. This demonstrates how character classes give precise control over what can appear at a position.

- * **Anchor testing on multiple strings:** The structured loop tests each string against both `^Plural` (must appear at the start) and `sight$` (must appear at the end). `'Pluralsight'` satisfies both; `'notPlural'` satisfies neither.
- * **re.escape for safe input handling:** The pattern `r'regex\.'` searches for `'regex'` followed by a literal period. `re.escape()` automates this escaping for any user-provided string.

Cell 3 - Repetition & Groups with Complete Analysis

```
# Cell 3: Quantifiers and groups with systematic analysis
import re

# Quantifier comparison table
quantifier_tests = [
    (r'.+\.txt', ['data.txt', 'notes.txt'], ['.txt']),
    (r'.*\.txt', ['data.txt', '.txt'], ['data']),
    (r'colou?r', ['color', 'colour'], ['colouur', 'colr']),
    (r'(ha){2}', ['haha', 'hahaha'], ['ha', 'hahaX']),
    (r'\d{4}', ['2024', '1234'], ['24', '123', '12345']),
]

for pattern, should_match, should_not in quantifier_tests:
    print(f'\n Pattern: {pattern!r}')
    for s in should_match:
        ok = bool(re.search(pattern, s))
        print(f'      {"Y" if ok else "UNEXPECTED"} Should match: {s!r}')
    for s in should_not:
        ok = bool(re.search(pattern, s))
        print(f'      {"N" if not ok else "UNEXPECTED"} Should NOT : {s!r}')

# Named groups: structured log parsing
log_lines = [
    '2024-11-15 10:32:45 ERROR Connection refused',
    '2024-11-15 10:33:01 INFO Retry attempt 1',
    '2024-11-15 10:33:15 ERROR Timeout after 30s',
]

log_pattern = re.compile(
    r'(?P<date>\d{4}-\d{2}-\d{2}) '
    r'\s+'
    r'(?P<time>\d{2}:\d{2}:\d{2}) '
    r'\s+'
    r'(?P<level>\w+) '
    r'\s+'
    r'(?P<message>.+)'
)

errors = []
for line in log_lines:
    match = log_pattern.fullmatch(line)
    if match:
        d = match.groupdict()
        if d['level'] == 'ERROR':
            errors.append(d)
```

```
print(f'    [{d["date"]} {d["time"]} {d["message"]}']')

print(f'\nExtracted {len(errors)} ERROR entries.')
```

Explanation

This cell delivers two key insights in one:

- * **Quantifier comparison as a truth table:** The test matrix for each quantifier explicitly documents which inputs should match and which should not. Observing that `.+` (one or more) rejects `.txt` (no characters before the dot) while `.*` (zero or more) accepts it gives quantifier semantics concrete meaning.
- * **Named group log parsing:** The `log_pattern` uses four named groups (date, time, level, message) to parse structured log lines. The `groupdict()` call returns a clean Python dictionary, making downstream processing (filtering by level, sorting by date) straightforward. This pattern is directly applicable to parsing ML training logs, API access logs, and pipeline execution traces.

Cell 4 - The Full re Module API

```
# Cell 4: Demonstrate every re module function
import re

text = 'A1B2C3'

# re.search() - first match anywhere
m = re.search(r'\d', text)
print(f'search: {m.group()!r} at span {m.span()}')

# re.match() - match at start only
print(f'match(A): {bool(re.match("A", text))}')
print(f'match(1): {bool(re.match("1", text))} (1 is not first char)')

# re.fullmatch() - entire string must match
print(f'fullmatch(digits only): {bool(re.fullmatch(r"[A-Z0-9]+", text))}')
print(f'fullmatch(digits only for 12345): {bool(re.fullmatch(r"\d+", "12345"))}')

# re.findall() - all matches as list
print(f'findall digits: {re.findall(r"\d", text)}')

# re.sub() - replace matches
print(f'sub digits with *: {re.sub(r"\d", "*", text)}')

# re.subn() - replace and count
result, count = re.subn(r'\d', '*', text)
print(f'subn result: {result!r}, substitutions: {count}')

# re.split() - split on pattern
print(f'split on whitespace: {re.split(r"\s+", "Python Regex Playbook")}')

```

```
# re.compile() - compiled pattern object
pattern = re.compile(r'\d+')
print(f'compiled findall: {pattern.findall("A1B22C333")}')

```

Explanation

This cell serves as a living API reference - every re module function demonstrated on the same input string, making direct comparison of their behaviours possible:

- * **search vs match vs fullmatch:** On 'A1B2C3': search finds '1' anywhere; match('A') succeeds (A is at start), match('1') fails (1 is not at start); fullmatch(r'\d+') fails (A is not a digit). These three functions address three different validation scenarios.
- * **sub vs subn:** Both replace all matches of r'\d' with '*', producing 'A*B*C*'. The difference is the return type: sub() returns just the new string; subn() returns a tuple (new_string, count). The count is useful for validation and analytics.
- * **compile() reuse:** The compiled pattern.findall('A1B22C333') demonstrates that \d+ with the + quantifier matches sequences of digits, not individual digits.

Cell 5 - Match Object Deep Dive

```
# Cell 5: Match object - all methods and attributes
import re

text = 'Age 25 and Score 98'

# Basic match object inspection
match = re.search(r'\d+', text)
print(f'group() : {match.group()!r}')
print(f'start() : {match.start()!r}')
print(f'end() : {match.end()!r}')
print(f'span() : {match.span()!r}')

# Verify span by slicing
start, end = match.span()
print(f'Slice verification: text[{start}:{end}] = {text[start:end]!r}')

# Named groups: the most readable approach
pattern = r'(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2}) '
date_text = 'Project deadline: 2024-11-15'
m = re.search(pattern, date_text)

print(f'group(0) : {m.group(0)!r}')
print(f'group(year) : {m.group("year")!r}')
print(f'groups() : {m.groups()!r}')
print(f'groupdict() : {m.groupdict()!r}')

# finditer: match objects for all matches
for m in re.finditer(r'\d+', text):
    print(f' Found {m.group()!r:6s} at positions {m.span()!r}')

# Build an annotation list

```

```

annotations = [
    {'text': m.group(), 'start': m.start(), 'end': m.end(), 'type':
    'NUMBER'}
    for m in re.finditer(r'\d+', text)
]
for ann in annotations:
    print(f'    {ann}')

```

Explanation

This cell demonstrates the full richness of the Match object - a tool that goes far beyond simply confirming 'yes, there is a match':

- * **span() and slice verification:** match.span() returns (4, 6) for '25' in 'Age 25 and Score 98'. The slice text[4:6] confirms '25'. This demonstrates that start() and end() are indices into the original string - essential for text annotation tasks.
- * **groupdict() for structured extraction:** Named groups with groupdict() return a Python dictionary. This is the cleanest possible interface between a regex match and downstream Python code: the field names are self-documenting, and the dict integrates naturally with pandas, JSON serialisation, and database insertion.
- * **finditer() for large-scale annotation:** re.finditer() returns a lazy iterator of Match objects rather than materialising all matches into a list. The list comprehension creates annotation dictionaries - a pattern used in NLP Named Entity Recognition (NER) systems.

Cell 6 - Lookahead & Lookbehind in Practice

```

# Cell 6: Lookaround assertions with progressive examples
import re

text = 'aB3cD4eF5gH6'

# Step 1: Find ALL uppercase letters
result_1 = re.findall(r'[A-Z]', text)
print(f'All uppercase      : {result_1}')

# Step 2: Uppercase letters PRECEDED BY lowercase (lookbehind)
result_2 = re.findall(r'(?<=[a-z])[A-Z]', text)
print(f'After lowercase     : {result_2}')

# Step 3: Uppercase preceded by lowercase AND FOLLOWED BY digit
result_3 = re.findall(r'(?<=[a-z])[A-Z](?=\d)', text)
print(f'After lower, before digit: {result_3}')

# Practical lookahead: extract keys from key: value pairs
config = 'model: RandomForest, n_estimators: 100, max_depth: 5, accuracy: 0.94'
keys = re.findall(r'[\w_]+(?:=)', config)
print(f'Keys: {keys}')

# Password validation with lookahead
password_pattern = re.compile(

```

```

    r'^'
    r'(?=.*[A-Z])'      # must contain at least one uppercase
    r'(?=.*[a-z])'      # must contain at least one lowercase
    r'(?=.*\d)'         # must contain at least one digit
    r'(?=.*[!@#$$%^&*])' # must contain at least one special char
    r'.{8,}'            # must be at least 8 characters
    r'$'
)

passwords = ['P@ssw0rd', 'password', 'PASSWORD1!', 'Valid@Pass9']
for pw in passwords:
    valid = bool(password_pattern.match(pw))
    print(f' {pw!r:20s} {"Valid" if valid else "Invalid"}')
```

Explanation

This cell builds lookahead comprehension progressively - from no lookahead to lookbehind to combined lookbehind+lookahead:

- * **Step-by-step construction:** Starting with 'all uppercase' and adding each constraint one at a time demonstrates exactly what each lookahead contributes. The progression method is the correct way to build complex patterns - add one constraint at a time, verify, then continue.
- * **Lookahead for key extraction:** The pattern `[w_]+(?:=)` extracts words immediately followed by a colon. The colon is asserted (lookahead) but not consumed - the match is only the key name.
- * **Password validation with multiple lookaheads:** Each `(?=...)` is an independent lookahead that starts from the beginning (^) and scans the entire string. All lookaheads must pass simultaneously for the match to succeed.

Cell 7 - Real-World Data Pipeline: Clean, Validate & Redact

```

# Cell 7: A complete data cleaning and validation pipeline
import re
from dataclasses import dataclass, field
from typing import Optional

# Pre-compiled patterns (compile once, use many times)
PATTERNS = {
    'email' : re.compile(r'^[a-zA-Z0-9._%+\-]@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$'),
    'phone_digits' : re.compile(r'\D'),
    'credit_card' : re.compile(r'\d{4}[\s-]\d{4}[\s-]\d{4}[\s-]\d{4}'),
    'whitespace' : re.compile(r'\s+'),
    'punctuation' : re.compile(r'^\w\s@.\-')
}

@dataclass
class CleanedRecord:
    raw_name: str
    raw_email: str
```

```
raw_phone:    str
raw_comment: str
clean_name:   str = ''
clean_email:  str = ''
clean_phone:  str = ''
clean_comment: str = ''
errors:       list = field(default_factory=list)

def process(self) -> 'CleanedRecord':
    # Clean name: normalise whitespace, strip punctuation
    name = PATTERNS['whitespace'].sub(' ', self.raw_name).strip()
    self.clean_name = PATTERNS['punctuation'].sub('', name)

    # Validate email
    email = self.raw_email.strip().lower()
    if PATTERNS['email'].fullmatch(email):
        self.clean_email = email
    else:
        self.errors.append(f'Invalid email: {email!r}')

    # Normalise phone: keep only digits
    digits = PATTERNS['phone_digits'].sub('', self.raw_phone)
    if len(digits) == 11 and digits[0] == '1':
        digits = digits[1:]
    if len(digits) == 10:
        self.clean_phone = f'({digits[:3]}) {digits[3:6]}-{
{digits[6:]}}'
    else:
        self.errors.append(f'Invalid phone: {self.raw_phone!r}')

    # Redact credit cards in comments
    self.clean_comment = PATTERNS['credit_card'].sub('****-****-****-
****',
self.raw_comment)
    return self
```

Explanation

This capstone cell assembles every technique introduced in the chapter into a production-grade data cleaning pipeline:

- * **Pre-compiled pattern dictionary (PATTERNS):** All patterns are compiled at module load time into a dictionary. This is the production pattern for any module that applies regex to many records: compile once at import, use repeatedly at runtime.
- * **Dataclass as processing unit:** CleanedRecord stores both raw and cleaned versions of each field, enabling before/after comparison and audit logging. The errors list accumulates validation failures without aborting processing.
- * **Phone normalisation pipeline:** The `r'\D'` pattern removes every non-digit character from the phone field regardless of format. The remaining digit string is then validated by length and reformatted into the canonical `(XXX) XXX-XXXX` format.
- * **Credit card redaction in free text:** The credit card pattern handles both hyphen-separated and space-separated formats, catching varied inputs correctly. This is the correct approach for sanitising user-generated content before storage or logging.

Chapter 12: Quick Reference - The Complete Regex Cheatsheet

Pattern / Syntax	Meaning	Example
<code>.</code>	Any char except newline	<code>f.n</code> -> fun, fan, fin
<code>^</code>	Start of string (or line with re.M)	<code>^Hello</code> -> matches 'Hello World'
<code>\$</code>	End of string (or line with re.M)	<code>World\$</code> -> matches 'Hello World'
<code>\.</code>	Literal dot (escaped)	<code>file\.txt</code> -> matches 'file.txt' not 'fileXtxt'
<code>[abc]</code>	One of a, b, or c	<code>[aeiou]</code> -> matches any vowel
<code>[a-z]</code>	Any lowercase letter	<code>[a-z]+</code> -> word in lowercase
<code>[^abc]</code>	Anything except a, b, c	<code>[^0-9]</code> -> non-digit
<code>\d</code>	Digit [0-9]	<code>\d{4}</code> -> '2024'
<code>\D</code>	Non-digit	<code>\D+</code> -> letters/symbols
<code>\w</code>	Word char [a-zA-Z0-9_]	<code>\w+</code> -> word token
<code>\s</code>	Whitespace [\t\n\r]	<code>\s+</code> -> one or more spaces
<code>(abc)</code>	Capturing group	<code>(\d{4})</code> -> captures year
<code>(?P<name>...)</code>	Named capturing group	<code>(?P<year>\d{4})</code> -> <code>match.group('year')</code>
<code>(?:...)</code>	Non-capturing group	<code>(?:http https)://</code>
<code>a b</code>	a OR b	<code>cat dog</code> -> 'cat' or 'dog'
<code>+</code>	One or more (greedy)	<code>\d+</code> -> '1', '42', '100'
<code>*</code>	Zero or more (greedy)	<code>\d*</code> -> '', '1', '42'
<code>?</code>	Zero or one (optional)	<code>colou?r</code> -> 'color' or 'colour'
<code>{n}</code>	Exactly n times	<code>\d{4}</code> -> exactly 4 digits
<code>{n,m}</code>	Between n and m times	<code>\d{2,4}</code> -> 2 to 4 digits
<code>+? *? ??</code>	Lazy (non-greedy) versions	<code><.*?></code> -> each tag individually
<code>(?=...)</code>	Positive lookahead	<code>\w+(?=:)</code> -> word before colon
<code>(?!...)</code>	Negative lookahead	<code>\d+(?!px)</code> -> number not followed by 'px'
<code>(?<=...)</code>	Positive lookbehind	<code>(?<=\\\$)[\d.]+</code> -> digits after \$
<code>(?<!=...)</code>	Negative lookbehind	<code>(?<!\d)\d{3}</code> -> 3 digits not after a digit

<code>\b</code>	Word boundary	<code>\bword\b -> whole word 'word'</code>
<code>re.I</code>	Case-insensitive flag	<code>re.search('py', 'Python', re.I)</code>
<code>re.M</code>	Multiline flag (^ and \$ per line)	<code>re.findall('^key', text, re.M)</code>
<code>re.S</code>	. matches newlines too	<code>re.search(r'<.*>', html, re.S)</code>
<code>re.X</code>	Verbose mode (whitespace/comments)	<code>re.compile(pattern, re.X)</code>

"Knowing regular expressions is the difference between spending 10 minutes on a data cleaning task and spending 10 hours on it." - Every data engineer who has learned regex

Regular expressions are one of the highest-leverage skills in the data science and GenAI practitioner's toolkit. The investment in learning regex syntax pays off immediately and repeatedly: faster data cleaning, more robust input validation, cleaner LLM output parsing, and safer sensitive data handling. Every concept in this chapter - from literal characters to named lookaheads - is production-applicable and battle-tested across millions of Python applications worldwide.